МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
”Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по лабораторной работе №6
по дисциплине
Системы искусственного интеллекта

Построение пайплайна одномерной регрессии

Студент:
Группа ИА-331 *И.А. Иванов*

Преподаватель:
К.И. Брагин

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ №6 4

1.1 Выбор набора данных 4

1.2 Теоретическое обоснование 4

1.3 Ход выполнения работы 5

1.3.1 Шаг 1: Загрузка и подготовка данных 5

1.3.2 Шаг 2: Разделение на обучающую и тестовую выборки 5

1.3.3 Шаг 3: Обучение модели линейной регрессии 6

1.3.4 Шаг 4: Оценка качества модели 6

1.3.5 Шаг 5: Визуализация результатов 7

1.3.6 Шаг 6: Применение модели для нового объекта 8

1.4 Анализ результатов 8

1.5 Контрольные вопросы 9

1.6 Листинг программы 10

ЦЕЛИ И ЗАДАЧИ

Цель лабораторной работы: разработка единого пайплайна для решения задачи одномерной регрессии.

Основные задачи:

- реализовать конвейер для выполнения всех стадий обработки данных при решении задачи одномерной регрессии;
- получение теоретических представлений о задаче регрессии;
- получение навыков использования пайплайна при решении задачи машинного обучения;
- получение навыков рефакторинга кода в задачах машинного обучения.

В качестве набора данных для выполнения индивидуального задания выбран набор данных **Abalone** (морское ушко), который содержит информацию о возрасте моллюсков, определяемом по количеству колец на раковине.

ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ №6

1.1 Выбор набора данных

Для выполнения лабораторной работы выбран набор данных **Abalone**, который содержит следующие характеристики:

- Количество записей: 4177
- Количество признаков: 8
- Категориальный признак: `Sex` (M — мужской, F — женский, I — детеныш)
- Числовые признаки: Length, Diameter, Height, Whole weight, Shucked weight, Viscera weight, Shell weight
- Целевая переменная: `Rings` (количество колец на раковине)

Для одномерной регрессии выбран признак `Length` (длина раковины), так как он является одним из основных измерений моллюска.

1.2 Теоретическое обоснование

Линейная регрессия — метод восстановления зависимости между двумя переменными. Пусть задана модель регрессии — параметрическое семейство функций $g(x, \alpha)$, где $\alpha \in \mathbb{R}^p$ — вектор параметров модели. Определим функционал качества аппроксимации целевой зависимости на выборке X^ℓ как сумму квадратов ошибок:

$$Q(\alpha, X^\ell) = \sum_{i=1}^{\ell} (g(x_i, \alpha) - y_i)^2$$

$$\alpha^* = \arg\min_{\alpha \in \mathbb{R}^p} Q(\alpha, X^\ell)$$

Стандартный способ решения этой оптимизационной задачи — воспользоваться необходимым условием минимума. Если функция $g(x, \alpha)$ достаточное число раз дифференцируема по α, то в точке минимума выполняется система p уравнений относительно p неизвестных:

$$\frac{\partial Q}{\partial \alpha}(\alpha, X^\ell) = 2\sum_{i=1}^{\ell}(g(x_i, \alpha) - y_i)\frac{\partial g}{\partial \alpha}(x_i, \alpha) = 0$$

С использованием библиотек машинного обучения эти формулы можно реализовать автоматически.

1.3 Ход выполнения работы

1.3.1 Шаг 1: Загрузка и подготовка данных

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv('abalone.csv')
print(f"           : {df.shape}")

X = df.drop('Rings', axis=1)
y = df['Rings']

feature = 'Length'
X_simple = X[[feature]]
print(f"                 : {feature}")
print(f"    5     : {X_simple[:5].values.ravel()}")
print(f"                 : {y[:5].values}")
```

Листинг 1.1 — Загрузка данных

Результат выполнения:

```
           : (4177, 9)
                 : Length
5      : [0.455 0.35  0.53  0.44  0.33 ]
                 : [15  7  9 10  7]
```

1.3.2 Шаг 2: Разделение на обучающую и тестовую выборки

```
X_train, X_test, y_train, y_test = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)
print(f"             : {X_train.shape}")
print(f"            : {X_test.shape}")
```

Листинг 1.2 — Разделение данных

Результат:

```
             : (3341, 1)
            : (836, 1)
```

Оптимальное соотношение тестовой и обучающей выборки составляет 20:80. Принцип разбиения: данные случайным образом разделяются на две непересекающиеся части; на обучающей выборке модель изучает закономерности; на тестовой выборке оценивается качество обобщения.

1.3.3 Шаг 3: Обучение модели линейной регрессии

```
regressor = LinearRegression()
regressor.fit(X_train, y_train)
print(f"         (): {regressor.coef_[0]:.4f}")
print(f"             (): {regressor.intercept_:.4f}")
```

Листинг 1.3 — Обучение модели

Результат:

```
(    ): 10.2280
  (      ): 2.9950
```

Метод `fit` класса `LinearRegression` вычисляет оптимальные коэффициенты модели с помощью метода наименьших квадратов, решая систему нормальных уравнений: $\beta = (X^T X)^{-1} X^T y$.

1.3.4 Шаг 4: Оценка качества модели

```
y_pred_train = regressor.predict(X_train)
y_pred_test = regressor.predict(X_test)

train_mse = mean_squared_error(y_train, y_pred_train)
test_mse = mean_squared_error(y_test, y_pred_test)
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)

print(f"\nMSE            : {train_mse:.4f}")
print(f"MSE            : {test_mse:.4f}")
print(f"R²             : {train_r2:.4f}")
print(f"R²             : {test_r2:.4f}")
```

Листинг 1.4 — Оценка качества

Результат:

```
MSE              : 7.0747
MSE              : 7.0486
R²               : 0.3312
R²               : 0.3493
```

1.3.5 Шаг 5: Визуализация результатов

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X_train, y_train, alpha=0.5, color='blue', label='
      ')
plt.plot(X_train, regressor.predict(X_train), color='red',
   linewidth=2, label='        ')
plt.xlabel('    (Length)')
plt.ylabel('          (Rings)')
plt.title('                    (     )')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.scatter(X_test, y_test, alpha=0.5, color='green', label='        ')
plt.plot(X_test, regressor.predict(X_test), color='red', linewidth=2,
   label='        ')
plt.xlabel('    (Length)')
plt.ylabel('          (Rings)')
```

```
plt.title('                    (     )')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('lab6_univariate_regression.png', dpi=150)
print("            'lab6_univariate_regression.png'")
```

Листинг 1.5 — Построение графиков

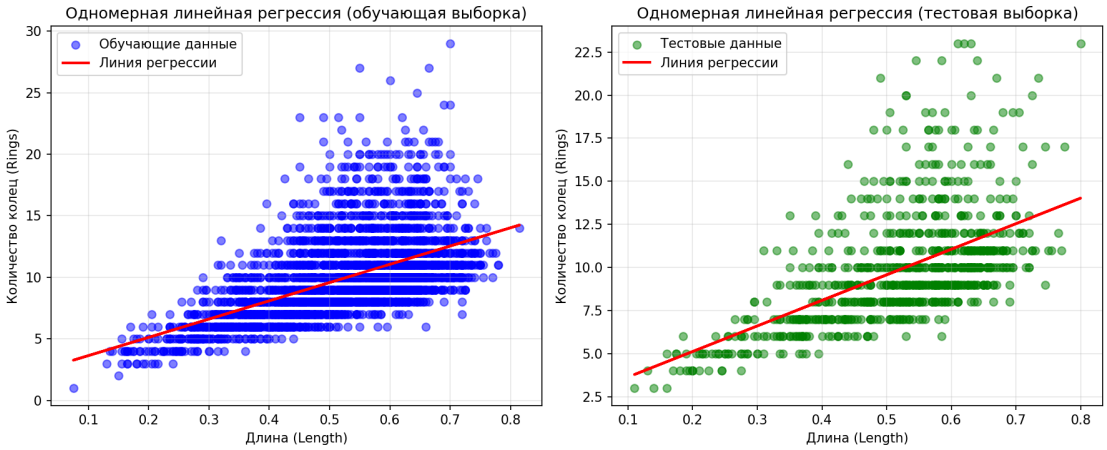

Рисунок 1 — Одномерная линейная регрессия: зависимость количества колец от длины раковины

1.3.6 Шаг 6: Применение модели для нового объекта

```
new_abalone = np.array([[0.55]])
prediction = regressor.predict(new_abalone)
print(f"\ n            0.55            : {prediction[0]:.2f}")
print(f"          : {int(round(prediction[0]))}")
```

Листинг 1.6 — Предсказание для нового объекта

Результат:

```
        0.55            : 8.62
    : 9
```

1.4 Анализ результатов

По результатам выполнения лабораторной работы можно сделать следующие выводы:

1. Одномерная регрессия с использованием признака `Length` показала низкое качество предсказания ($R^2 = 0.3493$ на тестовой выборке).
2. Коэффициент детерминации $R^2 = 0.3493$ означает, что модель объясняет только около 35% дисперсии целевой переменной.
3. Низкое качество модели объясняется слабой корреляцией между длиной раковины и количеством колец (коэффициент корреляции Пирсона составляет 0.5567).
4. Среднеквадратичная ошибка (MSE) на тестовой выборке составила 7.0486, что в среднем дает ошибку предсказания около 2.65 колец (корень из MSE).
5. Полученное уравнение регрессии: `Rings = 10.2280 × Length + 2.9950`.

1.5 Контрольные вопросы

1. Почему при реализации линейной модели регрессии нет необходимости выполнять масштабирование признаков?

При линейной регрессии масштабирование признаков не влияет на предсказательную способность модели, так как коэффициенты модели автоматически подстраиваются под масштаб каждого признака. Если признак увеличить в k раз, коэффициент уменьшится в k раз, и произведение останется неизменным. Однако масштабирование может быть полезно для ускорения сходимости градиентных методов и для интерпретации коэффициентов.

2. Почему при реализации модели линейной регрессии в качестве функции потерь используется квадратичное отклонение, а не модуль отклонения?

Квадратичное отклонение дифференцируемо, что позволяет использовать аналитические методы оптимизации (метод наименьших квадратов) и градиентный спуск. Кроме того, оно штрафует большие ошибки сильнее, что часто желательно на практике. Функция модуля отклонения не дифференцируема в нуле, что создает проблемы при оптимизации.

3. Что именно реализовано в методе fit(X, y) класса LinearRegression?

Метод `fit` вычисляет оптимальные коэффициенты модели с помощью метода наименьших квадратов (MSE), решая систему нормальных уравне-

ний: $\beta = (X^T X)^{-1} X^T y$. Для этого используется численно устойчивая реализация на основе сингулярного разложения (SVD).

4. Что такое p-значение? Как p-значение используется при оптимизации моделей регрессии?

P-значение — это вероятность получить наблюдаемый результат при условии, что нулевая гипотеза верна. В регрессии p-значение для коэффициента показывает, значимо ли отличается коэффициент от нуля. Обычно признаки с $p > 0.05$ считаются незначимыми и могут быть удалены из модели.

5. Поясните назначение метода predict класса LinearRegression.

Метод `predict` вычисляет предсказанные значения целевой переменной для новых данных на основе обученной модели: $\hat{y} = X\beta$. Для каждого объекта из входной матрицы X вычисляется линейная комбинация признаков с весами-коэффициентами.

6. Поясните назначение метода plot и scatter класса pyplot.

Метод `scatter` строит диаграмму рассеяния — отображает точки с координатами (x, y). Метод `plot` строит линейный график, соединяя точки отрезками. В данной работе `scatter` использовался для отображения реальных данных, а `plot` — для отображения линии регрессии.

7. По какой подвыборке необходимо оценивать точность модели машинного обучения: тестовой или тренировочной?

Точность модели необходимо оценивать по тестовой выборке, так как она не участвовала в обучении и позволяет оценить способность модели к обобщению на новых, ранее не виденных данных. Оценка по обучающей выборке может быть завышена из-за переобучения.

1.6 Листинг программы

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```python
from sklearn.pipeline import Pipeline

df = pd.read_csv('abalone.csv')
print(f"           : {df.shape}")

X = df.drop('Rings', axis=1)
y = df['Rings']

feature = 'Length'
X_simple = X[[feature]]
print(f"                 : {feature}")
print(f"    5     : {X_simple[:5].values.ravel()}")
print(f"                : {y[:5].values}")

X_train, X_test, y_train, y_test = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)
print(f"            : {X_train.shape}")
print(f"            : {X_test.shape}")

regressor = LinearRegression()
regressor.fit(X_train, y_train)
print(f"         (): {regressor.coef_[0]:.4f}")
print(f"            (): {regressor.intercept_:.4f}")

y_pred_train = regressor.predict(X_train)
y_pred_test = regressor.predict(X_test)

train_mse = mean_squared_error(y_train, y_pred_train)
test_mse = mean_squared_error(y_test, y_pred_test)
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)

print(f"\nMSE           : {train_mse:.4f}")
print(f"MSE           : {test_mse:.4f}")
print(f"R²            : {train_r2:.4f}")
print(f"R²            : {test_r2:.4f}")

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X_train, y_train, alpha=0.5, color='blue', label='
      ')
plt.plot(X_train, regressor.predict(X_train), color='red',
   linewidth=2, label='        ')
```

```
plt.xlabel('    (Length)')
plt.ylabel('          (Rings)')
plt.title('                    (     )')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.scatter(X_test, y_test, alpha=0.5, color='green', label='        ')
plt.plot(X_test, regressor.predict(X_test), color='red', linewidth=2,
    label='        ')
plt.xlabel('    (Length)')
plt.ylabel('          (Rings)')
plt.title('                    (     )')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('lab6_univariate_regression.png', dpi=150)
print("            'lab6_univariate_regression.png'")

new_abalone = np.array([[0.55]])
prediction = regressor.predict(new_abalone)
print(f"\ n            0.55            : {prediction[0]:.2f}")
print(f"          : {int(round(prediction[0]))}")
```

Листинг 1.7 — Полный код лабораторной работы №6

Вывод

В ходе выполнения лабораторной работы №6 был разработан пайплайн для решения задачи одномерной регрессии. На основе набора данных Abalone была построена модель линейной регрессии для предсказания количества колец по длине раковины.

Полученные результаты:

- Уравнение регрессии: `Rings = 10.2280 × Length + 2.9950`
- Коэффициент детерминации $R^2 = 0.3493$
- Среднеквадратичная ошибка MSE = 7.0486

Модель показала низкое качество предсказания, что объясняется слабой корреляцией между выбранным признаком и целевой переменной. Для

улучшения качества необходимо использовать многомерную регрессию с несколькими признаками. Все поставленные задачи выполнены.